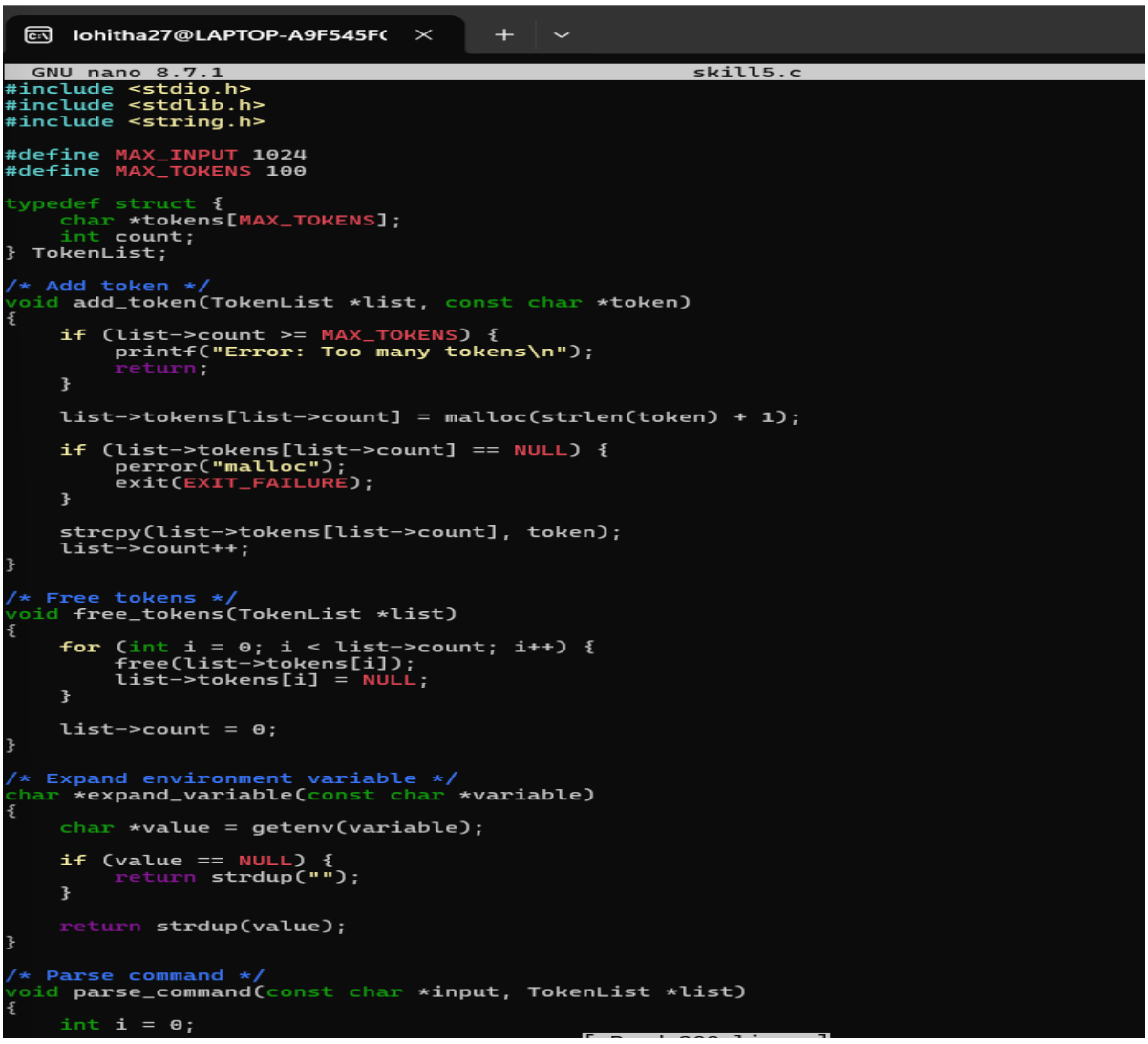
OS & SP

Name : S LOHITHA **Roll No :** 2520030109

SKILL – 05

1.Skill5.c



lohitha27@LAPTOP-A9F545F(

GNU nano 8.7.1 skill5.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

typedef struct {
    char *tokens[MAX_TOKENS];
    int count;
} TokenList;

/* Add token */
void add_token(TokenList *list, const char *token)
{
    if (list->count >= MAX_TOKENS) {
        printf("Error: Too many tokens\n");
        return;
    }

    list->tokens[list->count] = malloc(strlen(token) + 1);

    if (list->tokens[list->count] == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }

    strcpy(list->tokens[list->count], token);
    list->count++;
}

/* Free tokens */
void free_tokens(TokenList *list)
{
    for (int i = 0; i < list->count; i++) {
        free(list->tokens[i]);
        list->tokens[i] = NULL;
    }

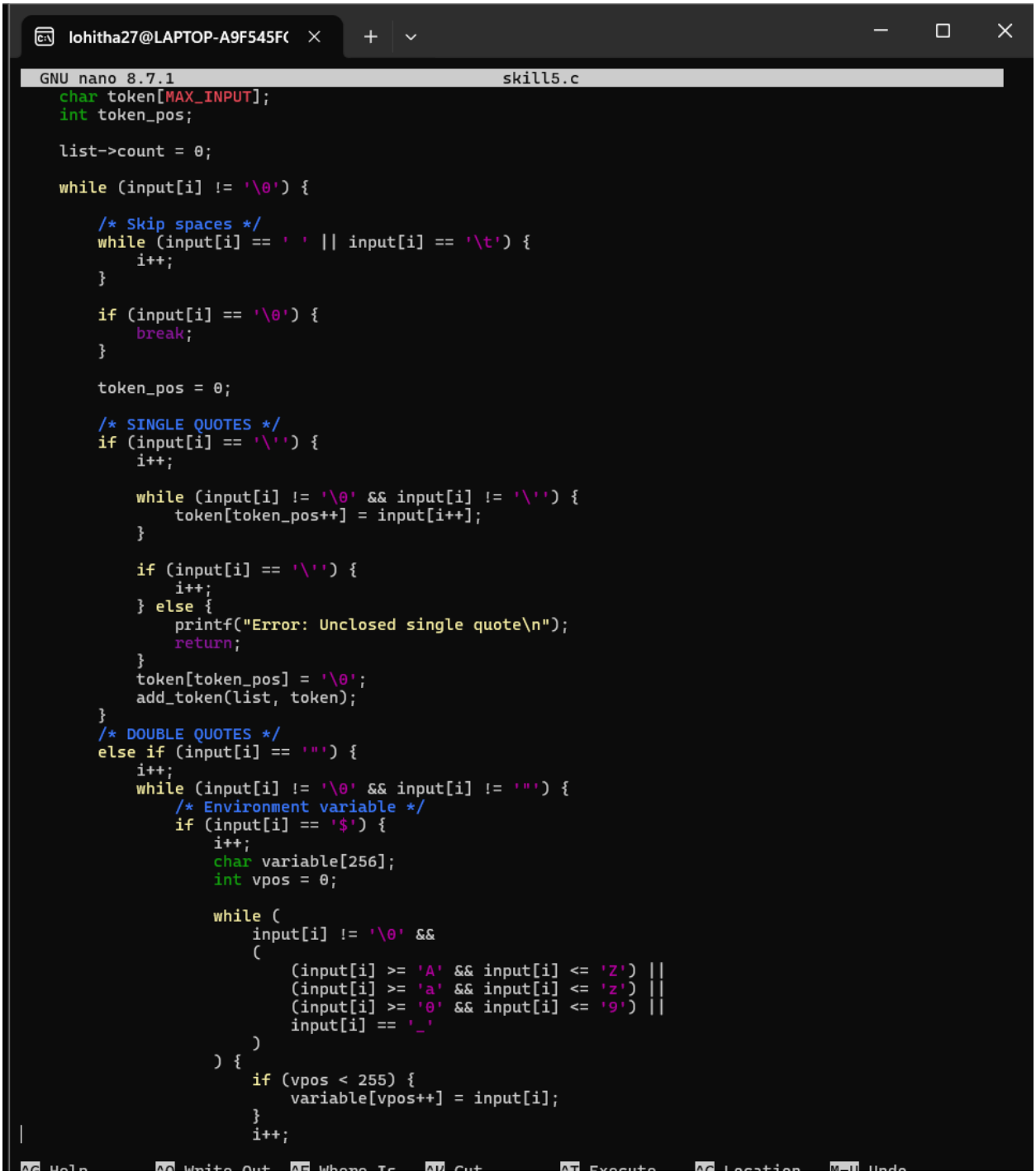
    list->count = 0;
}

/* Expand environment variable */
char *expand_variable(const char *variable)
{
    char *value = getenv(variable);

    if (value == NULL) {
        return strdup("");
    }

    return strdup(value);
}

/* Parse command */
void parse_command(const char *input, TokenList *list)
{
    int i = 0;
```


lohitha27@LAPTOP-A9F545F(
GNU nano 8.7.1 skill5.c
char token[MAX_INPUT];
int token_pos;
list->count = 0;
while (input[i] != '\0') {
/* Skip spaces */
while (input[i] == ' ' || input[i] == '\t') {
i++;
}
if (input[i] == '\0') {
break;
}
token_pos = 0;
/* SINGLE QUOTES */
if (input[i] == '\'') {
i++;
while (input[i] != '\0' && input[i] != '\'') {
token[token_pos++] = input[i++];
}
if (input[i] == '\'') {
i++;
} else {
printf("Error: Unclosed single quote\n");
return;
}
token[token_pos] = '\0';
add_token(list, token);
}
/* DOUBLE QUOTES */
else if (input[i] == '"') {
i++;
while (input[i] != '\0' && input[i] != '"') {
/* Environment variable */
if (input[i] == '$') {
i++;
char variable[256];
int vpos = 0;
while (
input[i] != '\0' &&
(
(input[i] >= 'A' && input[i] <= 'Z') ||
(input[i] >= 'a' && input[i] <= 'z') ||
(input[i] >= '0' && input[i] <= '9') ||
input[i] == '_'
)
) {
if (vpos < 255) {
variable[vpos++] = input[i];
}
i++;


lohitha27@LAPTOP-A9F545F(
GNU nano 8.7.1 skill5.c
}
variable[vpos] = '\0';
char *value = expand_variable(variable);
for (int j = 0; value[j] != '\0'; j++) {
if (token_pos < MAX_INPUT - 1) {
token[token_pos++] = value[j];
}
}
free(value);
} else {
if (token_pos < MAX_INPUT - 1) {
token[token_pos++] = input[i];
}
i++;
}
}
if (input[i] == '"') {
i++;
} else {
printf("Error: Unclosed double quote\n");
return;
}
token[token_pos] = '\0';
add_token(list, token);
}
/* NORMAL TOKEN */
else {
while (
input[i] != '\0' &&
input[i] != ' ' &&
input[i] != '\t'
) {
if (token_pos < MAX_INPUT - 1) {
token[token_pos++] = input[i];
}
i++;
}
token[token_pos] = '\0';
add_token(list, token);
}
}
}
/* Display tokens */
void print_tokens(const TokenList *list)
{
printf("\n--- Parsed Tokens ---\n");
for (int i = 0; i < list->count; i++) {
printf("Token %d: [%s]\n",
i + 1, list->tokens[i]);
}
printf("Total tokens: %d\n", list->count);
}
int main()
{
char input[MAX_INPUT];
TokenList list = { .count = 0 };
printf("====================================\n");

```
            i + 1, list->tokens[i]);
    }
    printf("Total tokens: %d\n", list->count);
}
int main()
{
    char input[MAX_INPUT];
    TokenList list = { .count = 0 };
    printf("====================================\n");
    printf("         OSSP PROGRAM 5\n");
    printf("         Quote Parser\n");
    printf("====================================\n");
    printf("Environment variable example: $USER\n");
    printf("Type 'exit' to quit.\n\n");
    while (1) {
        printf("parser> ");
        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }
        input[strcspn(input, "\n")] = '\0';
        if (strcmp(input, "exit") == 0) {
            break;
        }
        if (strlen(input) == 0) {
            printf("Error: Empty command\n\n");
            continue;
        }
        parse_command(input, &list);
        print_tokens(&list);
        free_tokens(&list);
        printf("\n");
    }
    printf("Parser terminated successfully.\n");
    return 0;
}

^G Help       ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit       ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

OUTPUT :

```
lohitha27@LAPTOP-A9F545FG:~/SKILL$ nano skill5.c
lohitha27@LAPTOP-A9F545FG:~/SKILL$ gcc skill5.c -o skill5
lohitha27@LAPTOP-A9F545FG:~/SKILL$ ./skill5
==================================
         OSSP PROGRAM 5
         Quote Parser
==================================
Environment variable example: $USER
Type 'exit' to quit.

parser> valgrind --leak-check=full --show-leak-kinds=all ./program5

--- Parsed Tokens ---
Token 1: [valgrind]
Token 2: [--leak-check=full]
Token 3: [--show-leak-kinds=all]
Token 4: [./program5]
Total tokens: 4

parser> echo 'hello world'

--- Parsed Tokens ---
Token 1: [echo]
Token 2: [hello world]
Total tokens: 2
```